

GLM-5 Cloud vs Qwen 3.5 vs Gemma 4 — App Modernisation Benchmark

Test Date: April 9, 2026

Models:

- qwen3.5:397b-cloud
- gemma4:31b-cloud
- glm-5:cloud

Tests (same suite as original appmod benchmark):

- Legacy PL/SQL Business Rules Extraction
- Java JEE Legacy Code Documentation
- OpenAPI Spec Generation from Requirements
- .NET WCF → .NET 8 Forward Engineering
- Integration Architecture Design (microservices decoupling)
- Test Specification from Business Rules

Speed Summary

Total Benchmark Time

Model	Total Time	vs Qwen
qwen3.5:397b-cloud	324.0s (5.4 min) ■	—
gemma4:31b-cloud	535.3s (8.9 min)	1.65x slower
glm-5:cloud	1005.9s (16.8 min)	3.1x slower

Per-test Timing

Test	Qwen 3.5	Gemma 4	GLM-5	Fastest
1. PL/SQL rules extraction	19.8s ■	47.2s	161.4s	Qwen (8.2x faster than GLM)
2. Java JEE documentation	18.1s ■	48.3s	109.1s	Qwen (6.0x faster)
3. OpenAPI spec generation	56.2s ■	101.5s	165.6s	Qwen (2.9x faster)
4. .NET forward engineering	114.4s	110.0s ■	144.7s	Gemma (marginal)
5. Integration design	58.6s ■	102.3s	318.2s	Qwen (5.4x faster)

Test	Qwen 3.5	Gemma 4	GLM-5	Fastest
6. Test specification	56.9s ■	126.0s	106.9s	Qwen (1.9x faster)

GLM-5's worst result: Integration Design at 318.2s — a 5.4x penalty vs Qwen for a task that Qwen handles cleanly in under a minute.

Output Depth (Word Count)

Test	Qwen 3.5	Gemma 4	GLM-5	Most Detailed
1. PL/SQL rules extraction	1,002	661	620	Qwen ■
2. Java JEE documentation	1,320	681	1,030	Qwen ■
3. OpenAPI spec generation	1,434	740	1,489	GLM-5 ■
4. .NET forward engineering	1,463	953	1,379	Qwen ■
5. Integration design	1,355	600	1,283	Qwen ■
6. Test specification	2,000	1,178	1,689	Qwen ■
Total	8,574	4,813	7,490	Qwen ■

Surprise finding: Qwen 3.5 is actually the most verbose model for app modernisation — it produces more detailed output *and* does it faster.

Quality Analysis by Test

Test 1 — PL/SQL Business Rules Extraction

Aspect	Qwen 3.5	Gemma 4	GLM-5
Rules identified	All 13+	~10	All 14 (with BR-IDs)
Format	Structured table	Prose + table	Clean table with Rule IDs
Hidden bugs caught	Flagged SLA breach note	Basic	■ Called out missing audit log on BR-06 (high-frequency escalation)
Output	1,002 words	661 words	620 words

Winner: Qwen for depth; GLM-5 for spotting the edge-case bug in BR-06 (missing audit log on frequency escalation path).

Test 2 — Java JEE Documentation

Aspect	Qwen 3.5	Gemma 4	GLM-5
Data flow coverage	■ Complete	Partial	■ Complete
Modernisation recommendations	■ Detailed	Basic	■ Detailed
JMS / transaction notes	■	■	■
Word count	1,320	681	1,030

Winner: Qwen (most detail, fastest). GLM solid but slower at 109s vs 18s.

Test 3 — OpenAPI 3.1 Spec Generation

Aspect	Qwen 3.5	Gemma 4	GLM-5
All 5 resources covered	■	Partial	■
JWT security scheme	■	■	■
Pagination headers	■	Limited	■
RFC 9457 Problem Details	■	✗	■
Rate limit headers	■	✗	■
Word count	1,434	740	1,489

Winner: GLM-5 edges Qwen on pure spec completeness (1,489 vs 1,434 words). Gemma significantly underperforms on this task — missing Problem Details and rate-limit headers entirely.

Test 4 — .NET WCF → .NET 8 Forward Engineering

Aspect	Qwen 3.5	Gemma 4	GLM-5
Minimal API vs controller choice	Recommends controller	Minimal API	Minimal API
EF Core included	■	■	■
FluentValidation	■	■	■
Result pattern	■	Partial	■
Serilog	■	■	■

Aspect	Qwen 3.5	Gemma 4	GLM-5
Async throughput	■	■	■
Program.cs registration	■ Complete	Partial	■ Complete
Speed	114.4s	110.0s ■	144.7s

Winner: Gemma4 (fastest, near-equal quality). This is the only test where Gemma beats Qwen on speed. GLM completes but adds ~35s of latency with no meaningful quality difference.

Test 5 — Integration Architecture Design

This is the **most complex appmod test** and where GLM-5 struggles the most (318.2s).

Aspect	Qwen 3.5	Gemma 4	GLM-5
Top 3 integrations identified	■	■	■
Event schemas (async)	■ Code examples	Limited	■ YAML + JSON
API contracts (sync)	■	Basic	■ OpenAPI snippets
Data consistency strategy	■ Saga/outbox	Partial	■ Outbox + compensating tx
Error handling / compensation	■	Basic	■ Detailed
Hybrid sync+async pattern justified	■	✗	■ Explicit rationale
Word count	1,355	600	1,283

Winner: Qwen narrowly (1,355 words vs 1,283 for GLM), with Qwen being 5.4x faster. Gemma's 600-word response is too shallow for a production integration design.

Test 6 — Test Specification from Business Rules

Aspect	Qwen 3.5	Gemma 4	GLM-5
All 13 rules covered	■	Most	■ All 13
Boundary value tests	■	Partial	■ Explicit boundary rows
Negative tests	■	■	■
Cross-rule interaction tests	■	Limited	■ Rich (fraud + threshold + freq)
Structured table format	■	■	■
Word count	2,000	1,178	1,689

Winner: Qwen (most complete at 2,000 words). GLM produces a well-structured spec with explicit boundary rows and cross-rule test scenarios — good quality just slower (106.9s vs 56.9s).

Summary Scorecard

Test	Winner	Notes
1. PL/SQL Rules Extraction	Qwen ■	GLM spotted hidden BR-06 audit bug
2. Java JEE Documentation	Qwen ■	6x faster, most detailed
3. OpenAPI Spec Generation	GLM-5 ■	Barely edges Qwen on completeness
4. .NET Forward Engineering	Gemma ■	Fastest, equal quality
5. Integration Design	Qwen ■	5.4x speed edge, equivalent depth
6. Test Specification	Qwen ■	2,000 words vs GLM's 1,689

Overall: Qwen 3.5 wins 4/6 tests. GLM-5 wins 1. Gemma 4 wins 1.

GLM-5 App Modernisation Verdict

■ Where GLM-5 Shines in App Modernisation

- **OpenAPI spec generation** — marginally best at producing complete, standards-compliant YAML (RFC 9457, rate headers, all resources)
- **PL/SQL bug detection** — called out the missing audit log on the high-frequency escalation path (the other models missed this)
- **Integration design quality** — solid hybrid sync/async justification with code examples
- **Test specification** — excellent cross-rule interaction scenarios with boundary value rows

■ Where It Fails for App Modernisation Work

- **3.1x slower overall** — a 16.8 minute benchmark run where Qwen does it in 5.4 minutes
- **Integration design is painfully slow** — 318.2s (5 min 18s) for a single task
- **Outputs less depth** — despite being slower, GLM-5 actually produces *fewer* words than Qwen in 5 of 6 tests
- **No clear quality advantage** to justify the latency in a consultant workflow

When to Use GLM-5 for App Modernisation

■ Good fit:

- Nightly spec validation tasks (OpenAPI completeness checks)
- One-shot PL/SQL audit where catching edge-case bugs matters
- Generating test specifications offline before a sprint

■ Not suitable:

- Live analysis sessions with a client
- Iterative forward engineering (too slow to iterate)
- Integration design workshops requiring quick responses

Recommendation

Keep `qwen3.5:397b-cloud` as the primary model for app modernisation work.

Qwen is faster, produces more words, and wins 4 out of 6 categories. The only two losses are razor-thin (OpenAPI spec) or a minor speed tie (.NET forward engineering).

GLM-5 has one genuine edge: **it's better at catching subtle logic gaps in legacy code** (the BR-06 audit log omission is a real defect that would surface in production). If you're doing a once-over compliance analysis of legacy PL/SQL, it's worth the wait. For everything else in the appmod workflow, Qwen 3.5 is the right choice.

Raw Result Files

All 18 output files saved in `./test-results-appmod/`:

File	Words	Time
<code>qwen3.5_397b-cloud_1_plsql_rules_extraction.md</code>	1,002	19.8s
<code>qwen3.5_397b-cloud_2_java_jee_documentation.md</code>	1,320	18.1s
<code>qwen3.5_397b-cloud_3_openapi_spec_generation.md</code>	1,434	56.2s
<code>qwen3.5_397b-cloud_4_dotnet_forward_engineering.md</code>	1,463	114.4s
<code>qwen3.5_397b-cloud_5_integration_design.md</code>	1,355	58.6s
<code>qwen3.5_397b-cloud_6_test_specification.md</code>	2,000	56.9s
<code>gemma4_31b-cloud_1_plsql_rules_extraction.md</code>	661	47.2s
<code>gemma4_31b-cloud_2_java_jee_documentation.md</code>	681	48.3s
<code>gemma4_31b-cloud_3_openapi_spec_generation.md</code>	740	101.5s
<code>gemma4_31b-cloud_4_dotnet_forward_engineering.md</code>	953	110.0s

File	Words	Time
gemma4_31b-cloud_5_integration_design.md	600	102.3s
gemma4_31b-cloud_6_test_specification.md	1,178	126.0s
glm-5_cloud_1_plsql_rules_extraction.md	620	161.4s
glm-5_cloud_2_java_jee_documentation.md	1,030	109.1s
glm-5_cloud_3_openapi_spec_generation.md	1,489	165.6s
glm-5_cloud_4_dotnet_forward_engineering.md	1,379	144.7s
glm-5_cloud_5_integration_design.md	1,283	318.2s
glm-5_cloud_6_test_specification.md	1,689	106.9s